



Architecture Documentation — TASCO Growth Platform

TASCO Insurance × VETC Growth Platform — Architecture

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Documents

System at a glance

Source map

Conventions used in these documents

Document	Version	Date	Owner	Status
Architecture documentation index	1.0	07 October 2026	iorta TechNXT Architecture	Issued for submission

TASCO Insurance × VETC motor insurance growth platform (new business, renewals, partners, claims FNOL), built by iorta TechNXT. Runtime: Node.js 22, PostgreSQL 16 (an in-memory store is available for development and demonstrations only; production refuses to start without `DATABASE_URL`), one runtime dependency (`pg`).

Every document in this set is written from the source code. Each document cites the files it describes, so a reviewer can check every claim. Where the platform currently uses a **sandbox** adapter rather than a production integration, the document says so. Regulatory statements are flagged **"confirm with TASCO legal"**.

Documents

#	Document	Audience	What it answers
1	Solution Architecture	CTO, architects, delivery leads	Principles, C4 views, hexagonal layering, key flows, event catalogue, configurability, scaling to 6 M vehicles, roadmap
2	Architecture decision records (see the ADR index below)	Architects, tech leads, auditors	The 12 decisions that shape the platform, with alternatives and consequences
3	Integration Architecture	Integration teams (VETC, TASCO core, Zalo, telco, voice - AI vendor, partners)	Every port: contract, protocol, authentication, idempotency, resilience, SLA, sandbox vs production
4	Deployment and Infrastructure Architecture	Platform/SRE, InfoSec, finance	Environments, Kubernetes topology, network zones, PostgreSQL HA, secrets, CI/CD, capacity and cost, observability
5	Data Architecture	Data office, DPO, data stewards, BI	ERD, data dictionary, MDM/golden record, data quality, lineage, retention, migration, analytics feed
6	Security Architecture	CISO, InfoSec, penetration testers, auditors	STRIDE threat model, OWASP Top 10 and ASVS L2 mapping, RBAC/ABAC, cryptography and keys, findings register
7	AI and Prompt Governance	Compliance, model risk, CX, product	Voice bot principles, script lifecycle, lead - score model risk, LLM policy, monitoring, RACI

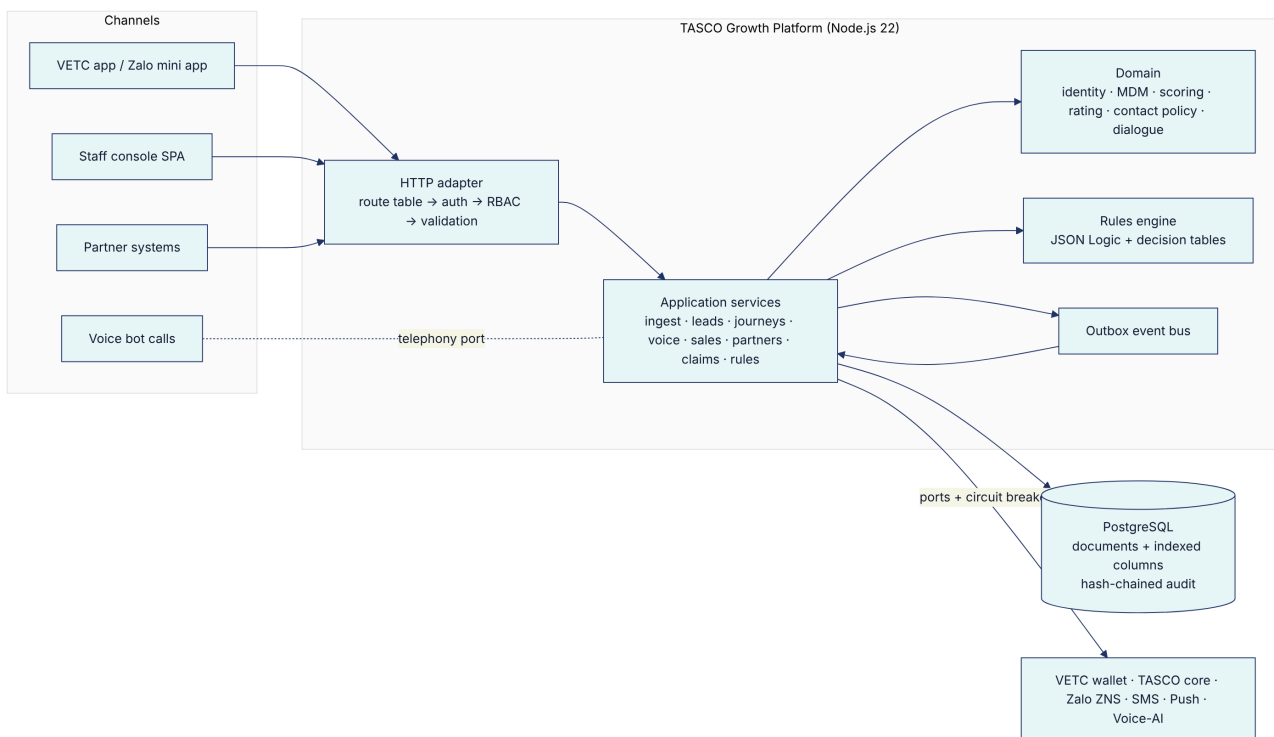
ADR index

ADR	Decision	Status
ADR-001	Hexagonal / Clean architecture	Accepted
ADR-002	Node.js 22, minimal dependencies (<code>pg</code> only at runtime), JavaScript + JSDoc	Accepted
ADR-003	JSON Logic rules engine with maker-checker governance	Accepted
ADR-004	PostgreSQL document + indexed-columns pattern; in-memory adapter with the same behaviour	Accepted
ADR-005	Outbox table + SKIP LOCKED relay now, message broker later	Accepted (gaps noted)
ADR-006	Field-level AES-256-GCM encryption + HMAC blind index	Accepted

ADR	Decision	Status
ADR-007	Local JWT + TOTP (self-enrolment, replay protection) now; OIDC federation to the TASCO IdP as target	Accepted (interim)
ADR-008	Hash-chained, append-only audit log	Accepted
ADR-009	Deterministic voice bot dialogue with a governed script; LLM only as an optional classifier behind a port	Accepted
ADR-010	API-first route table that generates OpenAPI	Accepted
ADR-011	Vanilla JS SPA + design tokens (no framework, strict CSP)	Accepted
ADR-012	Containers; Kubernetes as primary target; Railway for pilots and UAT	Accepted
ADR-013	TASCO core as the master for products and rating; CoreRating and ProductCatalogue ports	Accepted

System at a glance

The diagram shows the four channels, the layers inside the platform and the external systems reached through ports.



Source map

Layer	Path	Contents
Shared kernel	src/shared/	Config (12-factor, *_FILE secrets), logger (PII redaction), crypto, validation, resilience, metrics, errors, clock
Rules engine	src/rules/	jsonLogic.js (safe subset), decisionTable.js, validators.js (per-kind validation, copy guard, statutory commission caps)

Layer	Path	Contents
Domain	src/domain/	identity.js, enrichment.js, leads.js, rating.js, contactPolicy.js, voicebot.js
Application	src/application/	One service per use-case area, plus accessPolicy.js (RBAC + ABAC + masking) and rulesService.js (versioning, maker-checker)
Composition root	src/bootstrap/container.js	Chooses adapters, wires services, registers event subscribers
Persistence adapters	src/adapters/persistence/	schema.js (collection registry), schemaSql.js, codec.js, memoryStore.js, postgresStore.js, auditChain.js, query.js
Messaging adapter	src/adapters/messaging/outboxEventBus.js	Outbox publish / relay / drain
Integration adapters	src/adapters/integrations/	Sandbox VETC wallet, TASCO core and notification gateways (mockGateways.js), simulated caller, synthetic VETC source
HTTP adapter	src/adapters/http/	app.js (pipeline), routes.js (80 operations), router.js, openapi.js, security.js
Processes	src/server.js, src/jobs/cli.js	API server with outbox relay and graceful shutdown; batch jobs
Configuration	config/rules/*.json (21 rule kinds), config/security/rbac.json	Business rules seeded as version 1; RBAC, separation-of-duties pairs, restricted rule kinds
Schema	db/migrations/001_init.sql, 002_integrity_hardening.sql	19 collection tables + audit_log; integrity hardening (one active rule version per kind, TRUNCATE block on audit_log, outbox status index)
Deployment	Dockerfile, docker-compose.yml, deploy/k8s/, railway.json, .github/workflows/ci.yml	See the Deployment and Infrastructure Architecture

Conventions used in these documents

- **Implemented:** present in the code at the cited path.
- **Sandbox:** the port and the calling code are real; the adapter behind the port simulates the external system (`src/adapters/integrations/*`).
- **Planned / target:** a design commitment that is not in the code yet. Each one names an owner phase in the roadmap.
- **Infra:** a control delivered by the platform (Kubernetes, WAF, managed PostgreSQL, CI) rather than by application code.
- **Fixed:** a finding from an earlier review that has been remediated in code; the mechanism is named where the finding is listed.